TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA: CÔNG NGHỆ THÔNG TIN
BỘ MÔN: CÔNG NGHỆ PHẦN MỀM

ĐỀ THI VÀ BÀI LÀM

Tên học phần: Toán ứng dụng CNTT
Mã học phần: Hình thức thi: *Tự luận có giám sát*
Đề số: **Đ0002** Thời gian làm bài: 90 phút *(không kể thời gian chép/phát đề)*
Được sử dụng tài liệu khi làm bài.

Họ tên: Huỳnh Phước Phú **Lớp**:23T_Nhat2 **MSSV**:102230094

Sinh viên làm bài trực tiếp trên tệp này, lưu tệp với định dạng MSSV_HọTên.pdf và nộp bài thông qua MS Teams.

Câu 1 (*2.0 điểm*): Viết chương trình (có sử dụng hàm) thực hiện công việc sau, biết rằng N=8300:

a) *(1.0 điểm)* Tìm số lượng và liệt kê các số hoàn hảo nhỏ hơn N.

Trả lời: Dán code bên dưới:

```
#include <iostream>
#include <cmath>
using namespace std;

const int MAX = 10001;



void generateSieve(bool Sieve[]) {
   for(int i = 0; i < MAX; i++) Sieve[i] = true;
   Sieve[0] = Sieve[1] = false;
   for(int i = 2; i * i < MAX; i++) {
      if(Sieve[i]) {
         for(int j = i * i; j < MAX; j += i) {
            Sieve[j] = false;
         }
      }
   }
}

long long Sumfactor(int n, bool Sieve[]) {
   int factor[MAX] = {0};

   int prime = 2;
   while(n > 1) {
      if(Sieve[prime]) {
         while((n > 1) && (n % prime == 0)) {
            factor[prime]++;
            n = n / prime;
```

```
            }
        }
        prime++;
    }

    long long sum = 1;
    for(int i = 2; i < MAX; i++) {
        if(factor[i] != 0) {
            sum *= (pow(i, factor[i] + 1) - 1) / (i - 1);
        }
    }
    return sum;
}

bool perfectNumber(int n, bool Sieve[]) {
    if(n == Sumfactor(n, Sieve) - n) return true;
    else return false;
}

void listperfect(int n, bool Sieve[]) {
    int index = 0;
    for(int i = 1; i < n; i++) {
        if(perfectNumber(i, Sieve)) {
            cout << i << " ";
            index++;
        }
    }
    cout << "\nSo luong so hoan hao nho hon " << n << " la: " << index << endl;
}
```

```
int main() {
    int n;
    cout << "Nhap n: ";
    cin >> n;

    bool Sieve[MAX];
    generateSieve(Sieve);

    listperfect(n, Sieve);

    return 0;
}
```

Trả lời: Dán kết quả thực thi vào bên dưới:

```
Nhap n: 8300
6 28 496 8128
So luong so hoan hao nho hon 8300 la: 4
```

b) *(1.0 điểm)* Cho M là số hoàn hảo lớn nhất vừa tìm được. Tìm số nguyên tố gần M nhất.

Trả lời: Dán code vào bên dưới:

```
#include <iostream>
#include <cmath>
using namespace std;

const int MAX = 1000001;



void generateSieve(bool Sieve[]) {
    for(int i = 0; i < MAX; i++) Sieve[i] = true;
    Sieve[0] = Sieve[1] = false;
    for(int i = 2; i * i < MAX; i++) {
        if(Sieve[i]) {
```

```
            for(int j = i * i; j < MAX; j += i) {
                Sieve[j] = false;
            }
        }
    }
}



int closestPrime(int n, bool Sieve[]) {
    int i = n + 1;
    if (n < 2) return 2;

    while(i < MAX) {
        if(Sieve[i]) break;
        else i++;
    }



    int j = n - 1;
    while(j >= 2) {
        if(Sieve[j]) break;
        else j--;
    }

    if (abs(n - i) < abs(n - j)) return i;
    else if (abs(n - i) > abs(n - j)) return j;
    else {
        cout << "It has 2 closest primes: " << j << " and " << i << endl;
        return 0;
    }
```

```
}

int main() {
   int n;
   cout << "Nhap n: ";
   cin >> n;

   bool Sieve[MAX];
   generateSieve(Sieve);

   int result = closestPrime(n, Sieve);
   if (result != 0) {
      cout << "So nguyen to gan nhat voi " << n << " la: " << result << endl;
   }

   return 0;
}
```

Trả lời: Dán kết quả thực thi vào bên dưới:

```
Nhap n: 8128
So nguyen to gan nhat voi 8128 la: 8123
```

Câu 2 (*2.0 điểm*): Cho ma trận A. Viết chương trình (có sử dụng hàm) thực hiện phân rã ma trận A bằng phương pháp SVD.

Trả lời: Dán code vào bên dưới

```
#include<bits/stdc++.h>
#include"Eigen/Dense"
#include"Eigen/Eigenvalues"
using namespace std;
using namespace Eigen;
#define Matrix vector<vector<float>>
```

```
Matrix operator*(Matrix a, Matrix b) {
Matrix c;
if(a[0].size() != b.size())
return c;
c = Matrix(a.size(), vector<float>(b[0].size(), 0));
for(int i = 0; i < c.size(); i++)
for(int j = 0; j < c[i].size(); j++)
for(int k = 0; k < b.size(); k++)
c[i][j] += a[i][k] * b[k][j];
return c;
}
Matrix Tranpose(Matrix A) {
Matrix res(A[0].size(), vector<float>(A.size()));
for(int i = 0; i < A.size(); i++)
for(int j = 0; j < A[i].size(); j++)
res[j][i] = A[i][j];
return res;
}MatrixXf FromMatrix(Matrix A) {
MatrixXf res = MatrixXf::Zero(A.size(), A[0].size());
for(int i = 0; i < A.size(); i++)
for(int j = 0; j < A[i].size(); j++)
res(i, j) = A[i][j];
return res;
}
Matrix ToMatrix(MatrixXf A) {
Matrix res = Matrix(A.rows(), vector<float>(A.cols()));
for(int i = 0; i < res.size(); i++)
for(int j = 0; j < res[i].size(); j++)
res[i][j] = A(i, j);
return res;
```

```
}
void Print(Matrix A, string name = "") {
cout << name << ":\n";
for(int i = 0; i < A.size(); i++) {
for(int j = 0; j < A[i].size(); j++)
printf("%10.5f ", A[i][j]);
printf("\n");
}
printf("\n");
}
Matrix A;
void Input() {
int n, m;cout << "Matrix size n, m = ";
cin >> n >> m;
A = Matrix(n, vector<float>(m));
cout << "Matrix: \n";
for(int i = 0; i < n; i++)
for(int j = 0; j < m; j++)
cin >> A[i][j];
}
void SwapCol(Matrix &A, int i, int j) {
for(int k = 0; k < A.size(); k++)
swap(A[k][i], A[k][j]);
}
vector<Matrix> EigenDecompose(Matrix A) {
MatrixXf M = FromMatrix(A);
EigenSolver<MatrixXf> S(M);
Matrix P = ToMatrix(S.pseudoEigenvectors());
Matrix G = ToMatrix(S.pseudoEigenvalueMatrix());
for(int i = 0; i < G.size() - 1; i++)
```

```
for(int j = i + 1; j < G.size(); j++)
if(G[i][i] < G[j][j]) {
swap(G[i][i], G[j][j]);
SwapCol(P, i, j);
}
return {P, G};
}
vector<float> MatrixSolve(Matrix A) {
for(int i = 0; i < A.size(); i++) {int i_m = i, v_m = A[i][i];
for(int j = i + 1; j < A.size(); j++)
if(abs(A[j][i] > v_m)) {
v_m = A[j][i];
i_m = j;
}
if(i_m != i)
swap(A[i], A[i_m]);
for (int j = i + 1; j < A.size(); j++) {
double f = 0;
if(A[i][i] != 0)
f = A[j][i] / A[i][i];
for (int k = i + 1; k < A[j].size(); k++)
A[j][k] -= A[i][k] * f;
A[j][i] = 0;
}
}
for(int i = 0; i < A.size(); i++)
for(int j = 0; j < A.size(); j++)
if(isnan(A[i][j]))
A[i][j] = 0;
vector<float> X(A.size(), 1);
```

```
for(int i = 0; i < A.size(); i++) {
int j;
bool f = false;
for(j = 0; j < A.size(); j++)
if(A[i][j] != 0) {
f = true;
break;
}if(f)
X[j] = -i;
}
for(int i = X.size() - 1; i >= 0; i--)
if(X[i] <= 0) {
int r = -X[i];
X[i] = 0;
for(int j = i + 1; j < A[r].size(); j++)
X[i] -= X[j] * A[r][j] / A[r][i];
}
float s = 0;
for(int i = 0; i < X.size(); i++)
s += X[i] * X[i];
for(int i = 0; i < X.size(); i++)
X[i] /= sqrt(s);
return X;
}
vector<Matrix> SVDDecompose(Matrix A) {
Matrix A_T = Tranpose(A);
vector<Matrix> E = EigenDecompose(A_T * A);
Print(E[0], "Eigen Vector");
Print(E[1], "Eigen Values");
Matrix V = E[0];
```

```
Matrix S(A.size(), vector<float>(A[0].size()));
if(isnan(E[1][0][0]) || E[1][0][0] <= 0) {
printf("Khong the phan ra ma tran!\n");
return vector<Matrix>();
}
for(int i = 0; i < min(E[1].size(), S.size()); i++)for(int j = 0; j < min(E[1][i].size(),
S[i].size()); j++) {
if(E[1][i][j] <= 0)
S[i][j] = 0;
else
S[i][j] = sqrt(E[1][i][j]);
if(isnan(S[i][j]))
S[i][j] = 0;
}
Matrix U = A * V;
while(U[0].size() < A.size())
for(int i = 0; i < U.size(); i++)
U[i].push_back(0);
while(U[0].size() > A.size())
for(int i = 0; i < U.size(); i++)
U[i].pop_back();
for(int i = 0; i < U.size(); i++) {
if(S[i][i] != 0)
for(int j = 0; j < U.size(); j++)
U[j][i] /= S[i][i];
else
for(int j = 0; j < U.size(); j++)
U[j][i] /= S[i][i];
}
for(int i = 0; i < U.size(); i++) {
if(i < min(S.size(), S[0].size()) && S[i][i] != 0)
```

```
continue;
vector<float> v = MatrixSolve(Tranpose(U));
for(int j = 0; j < U.size(); j++)
U[j][i] = v[j];
}return {U, S, Tranpose(V)};
}
int main() {
Input();
vector<Matrix> SVD = SVDDecompose(A);
if(SVD.size() > 0) {
Print(SVD[0], "U");
Print(SVD[1], "Sigma");
Print(SVD[2], "V^T");
Print(SVD[0] * SVD[1] * SVD[2], "Validating U * Sigma * V^T");
}
}
```

Trả lời: Dán kết quả thực thi vào bên dưới biết rằng $A = \begin{bmatrix} 3 & 4 & 5 \\ 6 & 8 & 10 \\ 1 & 2 & 3 \end{bmatrix}$, sai số $\varepsilon = 10^{-5}$.

```
PS E:\HocTap\01_TOANUD> cd e:\HocTap\01_TOANUD\CuoiKi
PS E:\HocTap\01_TOANUD\CuoiKi> .\SVD_eigen
Matrix size n, m = 3 3
Matrix:
3 4 5
6 8 10
1 2 3
Eigen Vector:
   0.41642    0.81237    0.40825
   0.56454    0.12093   -0.81650
   0.71266   -0.57047    0.40825

Eigen Values:
 263.54465    0.00000    0.00000
   0.00000    0.45534    0.00000
   0.00000    0.00000   -0.00000

U:
   0.43555    0.10147    0.89443
   0.87110    0.20294   -0.44721
   0.22690   -0.97391    0.00000

Sigma:
  16.23406    0.00000    0.00000
   0.00000    0.67479    0.00000
   0.00000    0.00000    0.00000

V^T:
   0.41642    0.56454    0.71266
   0.81237    0.12093   -0.57047
   0.40825   -0.81650    0.40825

Validating U * Sigma * V^T:
   3.00000    4.00000    5.00000
   6.00000    8.00000   10.00000
   1.00000    2.00001    3.00000
```

Câu 3 (*3.0 điểm*): Cho 15 điểm trong không gian Oxy như sau: (3, 4); (5, 6); (7, 8); (9, 10); (9, 1); (5, 2); (6, 3); (7, 4); (3, 11); (1, 9);); (2, 11); (3, 8); (5, 12); (10, 3); (9, 6).

a) *(1.0 điểm) Mô tả thuật toán xác định bao lồi của tập điểm đã cho*

Trả lời: dán sơ đồ khối hoặc mã giả:

Thuật toán:

Bước 1 : Sắp xếp các điểm tăng dần theo thứ tự hoành độ x (nếu hoành độ x bằng nhau thì ưu tiên tung độ độ y) ⇒ Ta được dãy điểm sắp xếp từ p1 => pn

Bước 2 : Xác định điểm điểm bên phải xa nhất và bên trái xa nhất (điểm p1 và pn)

Bước 3 : ***Xác định bao đóng trên*** bằng cách thêm đỉnh tiếp theo vào bao đóng với hai điểm đầu tiên là điểm bên trái xa nhất (p1) và điểm tiếp theo (p2). Sau khi thêm vào bao đóng, ***nếu 3 điểm cuối cùng trong tập không tạo ra nhánh rẽ phải (tức là rẽ trái hoặc thẳng hàng)*** thì xoá đỉnh ở giữa. Lặp lại bước trên cho đến khi gặp điểm xa nhất bên phải (pn).

Bước 4 : ***Xác định bao đóng dưới*** bằng cách thêm đỉnh tiếp theo vào bao đóng với hai điểm đầu tiên là điểm bên phải xa nhất (pn) và điểm tiếp theo là p(n-1). Sau khi thêm vào bao đóng, ***nếu 3 điểm cuối cùng trong tập không tạo ra nhánh rẽ phải (tức là rẽ trái hoặc thẳng hàng)*** thì xoá đỉnh ở giữa. Lặp lại bước trên cho đến khi gặp điểm xa nhất bên trái (p1).

Bước 5: Xóa điểm đầu và điểm cuối của tập bao dưới, sau đó hợp tập các điểm bao dưới vào tập các điểm bao trên ta sẽ có danh sách các điểm bao lồi theo chiều kim đồng hồ.

Pseudo code:

Input: tập hợp các điểm trên mặt phẳng

Output: Danh sách các điểm tạo thành Convex Hull của tập điểm đầu vào

Các bước tiến hành:

1. Sort các điểm tăng dần theo thứ tự giá trị x ➔ Được chuỗi các điểm đã sắp xếp p1, p2 ,..., pn
2. Thêm p1 và p2 vào list Lupper với p1 là điểm bắt đầu

Xác định bao trên

3. **For** i ← 3 **to** n
4. **Do** thêm pi vào Lupper
5. **While** Lupper.size() > 2 **and** DET(3 điểm cuối cùng trong Lupper) > 0
6. **Do** xóa điểm giữa trong 3 điểm cuối cùng của Lupper

Xác định bao dưới

7. Thêm pn và p(n-1) vào Llower với pn là điểm bắt đầu
8. **For** i ← n-2 to 1
9. **Do** thêm pi vào Llower
10. **While** Llower.size() > 2 **and** DET(3 điểm cuối cùng trong Llower) > 0
11. **Do** xóa điểm giữa trong 3 điểm cuối cùng của Llower

12. Xóa điểm đầu và điểm cuối của Llower
13. Nối Lupper và Llower để tạo thành tập bao lồi L

Return L

```
hàm cross_product(Point o, Point A, Point B) {
        gán int x <- (A.x - o.x) * (B.y - o.y) - (A.y - o.y) * (B.x - O.x)
        nếu x = 0 thì trả về 0
        nếu x > 0 thì trả về 1
        trả về 2
    }
    Hàm convertHull(vector<Point> A) {
        Gán int n <- kích thước của A;
        nếu (n bé hơn 3) trả về A
        Gán vector<Point> hull <- vector<Point> trống;
        Gán int left <- 0
        For(i <- 1, i bé hơn n, i++) {
            nếu (A[i].x bé hơn A[left].x) thì gán left <- i
        }
        Gán int p <- left, q
        While(p không bằng left) {
            đưa vào hull A[p];
            gán p <- (p+1) % n;
            For(i <- 0, i bé hơn n, i++) {
                nếu (cross_product(A[p], A[i], A[q]) bằng 2) thì gán q <- i
                }
            gán p <- q;
        }
        trả về hull
    }
```

b) *(1.0 điểm)* Viết hàm xác định bao lồi và cạnh nhỏ nhất của đa giác lồi vừa tìm được

Trả lời: Dán code bên dưới:

```
#include <iostream>
#include <vector>
#include <cmath>
using namespace std;


class Point {
public:
   double x, y;
   Point(double x, double y) {
```

```
        this->x = x;
        this->y = y;
    }
    Point() {}
};

int cross_product(Point O, Point A, Point B) {
    double x = (A.x - O.x) * (B.y - O.y) - (A.y - O.y) * (B.x - O.x);
    if (x == 0) return 0;
    else if (x > 0) return 1;
    else return 2;
}

vector<Point> Convex_Hull(vector<Point> A) {
    int n = A.size();
    if (n < 3) return A;
    vector<Point> hull;

    int left = 0;
    for (int i = 1; i < n; i++) {
        if (A[i].x < A[left].x) {
            left = i;
        }
    }

    int p = left, q;
    do {
        hull.push_back(A[p]);
        q = (p + 1) % n;
        for (int i = 0; i < n; i++) {
```

```
            if (cross_product(A[p], A[i], A[q]) == 2) {
                q = i;
            }
        }
        p = q;
    } while (p != left);
    return hull;
}

double shortest(Point a, Point b) {
    return sqrt((a.x - b.x)*(a.x - b.x) + (a.y - b.y)*(a.y - b.y));
}

bool isSamePoint(Point a, Point b) {
    return fabs(a.x - b.x) < 1e-6 && fabs(a.y - b.y) < 1e-6;
}

vector<Point> elementsInAAndNotInB(vector<Point> A, vector<Point> B) {
    vector<Point> result;
    for (int i = 0; i < A.size(); i++) {
        bool found = false;
        for (int j = 0; j < B.size(); j++) {
            if (isSamePoint(A[i], B[j])) {
                found = true;
                break;
            }
        }
        if (!found) {
            result.push_back(A[i]);
        }
```

```
    }
    return result;
}

int main() {
    vector<Point> points;
    points.push_back({3, 4});
    points.push_back({5, 6});
    points.push_back({7, 8});
    points.push_back({9, 10});
    points.push_back({9, 1});
    points.push_back({5, 2});
    points.push_back({6, 3});
    points.push_back({7, 4});
    points.push_back({3, 11});
    points.push_back({1, 9});
    points.push_back({2, 11});
    points.push_back({3, 8});
    points.push_back({5, 12});
    points.push_back({10, 3});
    points.push_back({9, 6});

    vector<Point> ans = Convex_Hull(points);

    cout << "Convex Hull points:\n";
    for (int i = 0; i < ans.size(); i++)
        cout << "(" << ans[i].x << ", " << ans[i].y << ")\n";

    double shot = 100000;
    for (int i = 0; i < ans.size(); i++) {
```

```
        for (int j = i + 1; j < ans.size(); j++) {
            double dis = shortest(ans[i], ans[j]);
            if (dis < shot) {
                shot = dis;
            }
        }
    }
    cout << "Shortest distance in convex hull: " << shot << endl;

    vector<Point> nothull = elementsInAAndNotInB(points, ans);

    cout << "Points not in convex hull:\n";
    for (int i = 0; i < nothull.size(); i++) {
        cout << "(" << nothull[i].x << ", " << nothull[i].y << ")\n";
    }

    return 0;
}
```

Trả lời: Dán kết quả thực thi vào bên dưới:

```
Convex Hull points:
(1, 9)
(2, 11)
(5, 12)
(9, 10)
(10, 3)
(9, 1)
(5, 2)
(3, 4)
Shortest distance in convex hull: 2.23607
```

c) *(1.0 điểm)* Xác định số lượng các điểm nằm bên trong bao lồi và liệt kê chúng

Trả lời: Dán code bên dưới:

```
#include <iostream>
#include <vector>
```

```
#include <cmath>
using namespace std;

class Point {
public:
    double x, y;
    Point(double x, double y) {
        this->x = x;
        this->y = y;
    }
    Point() {}
};

int cross_product(Point O, Point A, Point B) {
    double x = (A.x - O.x) * (B.y - O.y) - (A.y - O.y) * (B.x - O.x);
    if (x == 0) return 0;
    else if (x > 0) return 1;
    else return 2;
}

vector<Point> Convex_Hull(vector<Point> A) {
    int n = A.size();
    if (n < 3) return A;
    vector<Point> hull;

    int left = 0;
    for (int i = 1; i < n; i++) {
        if (A[i].x < A[left].x) {
            left = i;
        }
```

```
    }

    int p = left, q;
    do {
        hull.push_back(A[p]);
        q = (p + 1) % n;
        for (int i = 0; i < n; i++) {
            if (cross_product(A[p], A[i], A[q]) == 2) {
                q = i;
            }
        }
        p = q;
    } while (p != left);
    return hull;
}

double shortest(Point a, Point b) {
    return sqrt((a.x - b.x)*(a.x - b.x) + (a.y - b.y)*(a.y - b.y));
}

bool isSamePoint(Point a, Point b) {
    return fabs(a.x - b.x) < 1e-6 && fabs(a.y - b.y) < 1e-6;
}

vector<Point> elementsInAAndNotInB(vector<Point> A, vector<Point> B) {
    vector<Point> result;
    for (int i = 0; i < A.size(); i++) {
        bool found = false;
        for (int j = 0; j < B.size(); j++) {
            if (isSamePoint(A[i], B[j])) {
```

```
            found = true;
            break;
        }
    }
    if (!found) {
        result.push_back(A[i]);
    }
  }
  return result;
}

int main() {
  vector<Point> points;
  points.push_back({3, 4});
  points.push_back({5, 6});
  points.push_back({7, 8});
  points.push_back({9, 10});
  points.push_back({9, 1});
  points.push_back({5, 2});
  points.push_back({6, 3});
  points.push_back({7, 4});
  points.push_back({3, 11});
  points.push_back({1, 9});
  points.push_back({2, 11});
  points.push_back({3, 8});
  points.push_back({5, 12});
  points.push_back({10, 3});
  points.push_back({9, 6});

  vector<Point> ans = Convex_Hull(points);
```

```
    vector<Point> nothull = elementsInAAndNotInB(points, ans);

    cout << "Points not in convex hull:\n";
    for (int i = 0; i < nothull.size(); i++) {
        cout << "(" << nothull[i].x << ", " << nothull[i].y << ")\n";
    }

    return 0;
}
```

Trả lời: Dán kết quả thực thi vào bên dưới:

```
Points not in convex hull:
(5, 6)
(7, 8)
(6, 3)
(7, 4)
(3, 11)
(3, 8)
(9, 6)
```

Câu 4 (*2.0 điểm*): Cho hàm số $f(x) = \frac{4e^{2x} + 8x^2 + 8x}{45 - x} - 15x$.

a) *(1.0 điểm) Trình bày thuật toán tối ưu hàm số đã cho* sử dụng phương pháp *gradient descent với momentum*, biết rằng tham số học (learning rate) γ, hệ số động lượng là α .

Trả lời: dán sơ đồ khối hoặc mã giả:

- Điều kiện dừng của Gradient Descent
  - Giới hạn số bước lặp
  - So sánh giá trị hàm của nghiệm tại 2 lần cập nhật
  - Kiểm tra giá trị tuyệt đối của Gradient
- Tốc độ hội tụ của Gradient Descent:

- Phụ thuộc vào điểm khởi tạo
- Phụ thuộc vào chỉ số học learning rate

Các bước tiến hành:

Input:

- Hàm f(x): Hàm số cần tối ưu.
- Giá trị ban đầu của x: Giá trị x ban đầu để bắt đầu quá trình tối ưu.
- Tham số học (learning rate): Tốc độ học tập, xác định độ lớn của bước nhảy khi cập nhật giá trị của x.
- Điều kiện dừng: Điều kiện dừng quá trình tối ưu, ví dụ như số lần lặp tối đa hoặc độ chính xác đạt yêu cầu.
- Hệ số động lượng (momentum coefficient)
- Sai số epslion

Output: Trả về giá trị tọa độ cuối cùng đạt giá trị tối ưu x và giá trị tối ưu f(x)

Bước 1: Khởi tạo một giá trị ban đầu cho x, thường là ngẫu nhiên hoặc có thể lựa chọn thủ công.
Bước 2: Lặp lại quá trình sau cho đến khi đạt được điều kiện dừng (ví dụ: số lần lặp tối đa hoặc đạt đủ độ chính xác):
a. Tính giá trị đạo hàm của hàm f'(x) tại điểm x hiện tại.
b. Cập nhật giá trị của x theo công thức: x = x - learning_rate * f'(x), trong đó learning_rate là tham số học.
c. Lặp lại bước a và b cho đến khi đạt được điều kiện dừng.
Bước 3: Trả về giá trị x cuối cùng là tọa độ mà f(x) đạt được giá trị tối ưu
Bước 4: Trả về giá trị tối ưu của f(x)

```
hàm func(double x) {
    trả về hàm f(x)
}
hàm grad(double x) {
    trả về đạo hàm f(x)
}

Hàm GB_momentum(double x) {
Gán double deltax <- 0
Gán double deltaX_new <- 0
```

```
gasn double x_new <- 0
For (i <- 1, i <- 10000, i++) {
    deltaX_new <- -GAMMA * grad(x) + alpha * deltaX
    x_new <- x - GAMMA * grad(x) + alpha * deltaX_new;
    deltaX <- deltaX_new;

    Nếu (trị tuyệt đối (x - x_new) lớn hơn sai số yêu cầu) {
        break;
    } còn không {
        x <- x_new;
    }
}
In ra "Min x = " + x;
}
```

b) *(1.0 điểm)* Viết chương trình (có dùng hàm) tính giá trị bé nhất của f(x) sử dụng phương pháp *gradient descent với momentum* với số bước lặp *N* và sai số ε.

```
# Trả lời: Dán code vào bên dưới:
#include <iostream>
#include <cmath>

#define EPSILON 1e-5
#define GAMMA 0.001
#define N 1000
#define alpha 0.1

using namespace std;

double func(double x)
{
```

```
   return (4*exp(2*x) + 8*x*x + 8*x) / (45 - x) - 15*x;


}


double grad(double x)
{
   return ((45 - x)* (8*exp(2*x) + 16 * x + 8) + 4*exp(2*x) + 8 * x * x + 8 * x) / pow(45 - x,
2) - 15;


}


double GD_momentum(double x)
{
   double deltaX = 0;
   double deltaX_new = 0;
   double x_new = 0;


   for (int i = 1; i < N; i++)
   {
      deltaX_new = -GAMMA * grad(x) + alpha * deltaX;
      x_new = x - GAMMA * grad(x) + alpha * deltaX_new;
      deltaX = deltaX_new;
      if (fabs(x - x_new) < EPSILON)
      {
         break;
      }
      else
      {
         x = x_new;
      }
   }
```

```
    cout << "Min x = " << x << endl;
    return func(x);
}

int main()
{
    double x = 0;
    cout << "f(x) = (4e^(2x) +8(x^2) +8x) / (45 - x) - 15x" << endl;

    double minfX = GD_momentum(x);
    cout << "Min f(x): " << minfX << endl;

    return 0;
}
```

Trả lời: **Dán kết quả thực thi** với điểm khởi $x = 0$, tham số học học (*learning rate*) $\gamma = 0.001$, hệ số động lượng (*momentum coefficient*) là $\alpha = 0.1$, số bước lặp $N \geq 1000$ và sai số $\varepsilon = 10^{-5}$:

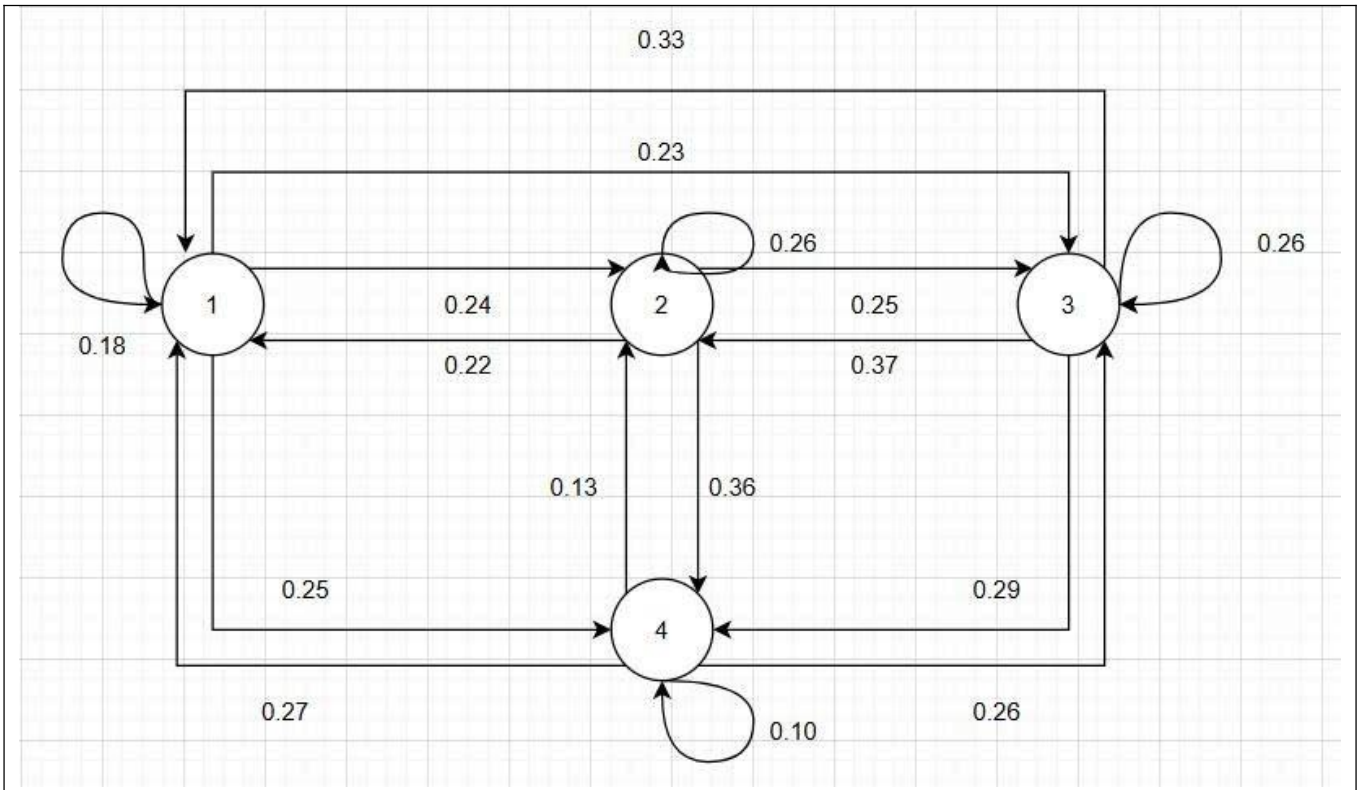
```
f(x) = (4e^(2x) +8(x^2) +8x) / (45 - x) - 15x
Min x = 2.15181
Min f(x): -24.1059
```

Câu 5 (*1.0 điểm*): Một hệ thống có chế độ làm việc ở mỗi giai đoạn vận hành chỉ với các trạng thái từ 1 đến 4. Chế độ làm việc của hệ thống này được mô tả bằng ma trận chuyển như sau:

$$P = \begin{bmatrix} 0.18 & 0.22 & 0.33 & 0.27 \\ 0.24 & 0.26 & 0.37 & 0.13 \\ 0.23 & 0.25 & 0.26 & 0.26 \\ 0.25 & 0.36 & 0.29 & 0.10 \end{bmatrix}$$

a) (*0.5 điểm*) Vẽ đồ thị biểu diễn chuỗi Markov tương ứng đã cho

Trả lời: Dán kết quả vào bên dưới



b) (*0.5 điểm*) Giả sử rằng hệ thống bắt đầu học ở trạng thái 1. Tính xác xuất hệ thống làm việc ở trạng thái 3 *sau ba và bốn bước thời gian vận hành*.

Trả lời: **Dán kết quả tính toán vào bên dưới:**

$$\boldsymbol{\pi(n+1) = \pi(n) \text{ x } P = \pi(0) \text{ x } p^{n+1}}$$

a) Ta có ma trận chuyển đổi trạng thái P sau 3 giai đoạn vận hành là:

$$p^3 = \begin{bmatrix} 0.18 & 0.22 & 0.33 & 0.27 \\ 0.24 & 0.26 & 0.37 & 0.13 \\ 0.23 & 0.25 & 0.26 & 0.26 \\ 0.25 & 0.36 & 0.29 & 0.10 \end{bmatrix} \times \begin{bmatrix} 0.18 & 0.22 & 0.33 & 0.27 \\ 0.24 & 0.26 & 0.37 & 0.13 \\ 0.23 & 0.25 & 0.26 & 0.26 \\ 0.25 & 0.36 & 0.29 & 0.10 \end{bmatrix} \times \begin{bmatrix} 0.18 & 0.22 & 0.33 & 0.27 \\ 0.24 & 0.26 & 0.37 & 0.13 \\ 0.23 & 0.25 & 0.26 & 0.26 \\ 0.25 & 0.36 & 0.29 & 0.10 \end{bmatrix}$$

$$= \begin{bmatrix} 0.225 & 0.267 & 0.312 & 0.160 \\ 0.226 & 0.269 & 0.310 & 0.195 \\ 0.225 & 0.267 & 0.312 & 0.197 \\ 0.225 & 0.268 & 0.310 & 0.197 \end{bmatrix}$$

Ta có vector phân phối xác xuất tại thời điểm bắt đầu làm việc là:

$\pi(0) = [1 \quad 0 \quad 0 \quad 0]$ (vì hệ thống bắt đầu ở trạng thái 1)

Vậy vector phân phối xác xuất sau giai đoạn vận hành là:

$$\pi(3) = \pi(0) \text{ x } p^3 = [1 \quad 0 \quad 0 \quad 0] \, x \begin{bmatrix} 0.225 & 0.267 & 0.312 & 0.160 \\ 0.226 & 0.269 & 0.310 & 0.195 \\ 0.225 & 0.267 & 0.312 & 0.197 \\ 0.225 & 0.268 & 0.310 & 0.197 \end{bmatrix} = [0.225 \quad 0.267 \quad 0.312 \quad 0.16]$$

Vậy xác xuất lớn nhất hệ thốngt làm việc ở trạng thái 4 sau ba giai đoạn vận hành nếu hệ thống bắt đầu làm việc ở trạng thái 1 là 0.312 (31,2%)

b) Ta có ma trận chuyển đổi trạng thái P sau 4 giai đoạn vận hành là:

$$p^4 = \begin{bmatrix} 0.225 & 0.267 & 0.311 & 0.196 \\ 0.225 & 0.267 & 0.311 & 0.196 \\ 0.225 & 0.268 & 0.311 & 0.196 \\ 0.225 & 0.268 & 0.311 & 0.196 \end{bmatrix}$$

Ta có vector phân phối xác xuất tại thời điểm bắt đầu làm việc là:

$\pi(0) = [1 \quad 0 \quad 0 \quad 0]$ (vì hệ thống bắt đầu ở trạng thái 1)

Vậy vector phân phối xác xuất sau giai đoạn vận hành là:

$$\pi(4) = \pi(0) \text{ x } p^4 = [1 \quad 0 \quad 0 \quad 0] \; x \begin{bmatrix} 0.225 & 0.267 & 0.311 & 0.196 \\ 0.225 & 0.267 & 0.311 & 0.196 \\ 0.225 & 0.268 & 0.311 & 0.196 \\ 0.225 & 0.268 & 0.311 & 0.196 \end{bmatrix} = [0.225 \quad 0.267 \quad 0.311 \quad 0.196]$$

Vậy xác xuất lớn nhất hệ thốngt làm việc ở trạng thái 4 sau ba giai đoạn vận hành nếu hệ thống bắt đầu làm việc ở trạng thái 1 là 0.311 (31,1%)

Đà Nẵng, ngày tháng 05 năm 2025

GIẢNG VIÊN BIÊN SOẠN ĐỀ THI **TRƯỞNG BỘ MÔN**